MTL783: Theory of Computation	1st Semester, 2025-2026
Lecture 8 — 22/09/2025	
Lecturer: Prof Minati De	Scribe: Team 16

Scribed by:

1. Arushi (2022MT11950)
2. Nikita Vikal (2022MT11951)
3. Rashi (2022MT11956)
4. Rashmi (2022MT11957)
5. Vaishali Chaudhary (2022MT11948)

Turing Machine Models and Equivalence

Turing Machine Recap

- M: Represents a Turing Machine.
- $L(M)$: Denotes the language of the machine M, which is the set of all strings that M accepts.

A language L is **Turing-recognizable** if there is a Turing Machine M that recognizes it. This means for any string w in the language L, the machine M will halt and accept. For strings not in L, M may halt and reject, or it may loop forever.

Decider

A language L is **Turing-decidable** if there exists a Turing Machine that decides it. A TM that decides a language is called a **decider**. It is a machine that is guaranteed to halt on all inputs; it will always either accept or reject.

A standard Turing Machine can be formally described as a 7-tuple:

$$M = (Q, \Sigma, \Gamma, \delta, q_0, q_{\text{accept}}, q_{\text{reject}})$$

Its behavior is governed by the transition function:

$$\delta : Q \times \Gamma \to Q \times \Gamma \times \{L, R\}$$

Models of Turing Machines

Model 1: Standard TM

This model consists of a control unit (which holds the current state), a read/write head, and a single tape that is infinite in one direction. The input is written on the tape, and the machine reads a symbol, writes a new symbol, and moves its head either left or right.

$$\cdots \mid a \mid b \mid c \mid \cdots \quad \text{(Infinite Tape)}$$

$$\uparrow$$

$$\text{(Read/Write Head)}$$

$$\uparrow$$

$$[\text{ Control Unit }] \quad \text{(State: } q_i\text{)}$$

Model 2: TM with a "Stay" Option

This is a variation where the read/write head can go left, right, or stay in the same position.

- These two models (Model 1 and Model 2) are equivalent. They recognize and decide the exact same set of languages.
- A "Stay" move can be simulated by a standard TM with two transitions: one move right followed by one move left.

Model 3: Multi-tape TM

This model uses multiple tapes, each with its own independent read/write head. The input string is placed on the first tape, and the others start blank.

$$\text{Tape 1: } \cdots |w| \cdots$$

$$\text{Tape 2: } \cdots | \ | \cdots$$

$$\text{Tape 3: } \cdots | \ | \cdots$$

$$\uparrow$$

$$[\text{ Control Unit }]$$

Equivalence of Models

Q: Does a multi-tape TM recognize more languages than a single-tape TM?
A: No.

Q: Can we create an equivalent single-tape TM for any multi-tape TM?
A: Yes.

Q: How?
A: We can simulate a multi-tape TM using a standard single-tape machine. The idea is to format the single tape to represent all of the tapes from the multi-tape machine.

We use a special separator symbol, like #, to distinguish the content of each tape. For example, the contents of three separate tapes can be stored on one tape like this:

$$\cdots \# \text{ content of tape 1 } \# \text{ content of tape 2 } \# \text{ content of tape 3 } \# \cdots$$

To keep track of each head's position, we can use a special marker or modify the symbols (e.g., use $\dot{a}$ to show the head is on symbol a). The single-tape TM works by sweeping back and forth across its tape to read the symbols under each virtual head and then sweeping back again to update the symbols and head positions according to the multi-tape machine's transition rule.

Although this simulation is much slower, it proves that a single-tape TM has the same computational power as a multi-tape TM.

Theorem

A language L is Turing-recognizable if and only if there exists a multitape Turing Machine that recognizes it. That is,

$$L = L(M).$$

So, additional tapes do not improve the powers of the machine; they only improve efficiency.

Non-deterministic Turing Machine (NDTM)

A non-deterministic Turing Machine is represented as:

$$TM = (Q, \Sigma, \Gamma, \delta, q_0, q_{\text{accept}}, q_{\text{reject}})$$

The difference lies in the transition function δ.

After reading the same symbol, we might have different states to go to, unlike the deterministic case where there is exactly one option.

Example:

- For a deterministic Turing Machine:

$$\delta : Q \times \Gamma \to Q \times \Gamma \times \{L, R\}$$

  Each (state, symbol) pair leads to a single next action.

- For a non-deterministic Turing Machine:

$$\delta : Q \times \Gamma \to \mathcal{P}(Q \times \Gamma \times \{L, R\})$$

  Here, δ may map to a set of possible next actions, meaning the machine can "branch" into multiple computational paths simultaneously.

Question:

Is a non-deterministic Turing Machine more powerful? Why? How?

Expansion:

- In terms of language recognition power, the answer is **No** — nondeterministic and deterministic Turing Machines recognize exactly the same class of languages.
- However, in terms of **efficiency**, nondeterminism might make certain computations faster (though this is a matter related to complexity theory, e.g., P vs NP).

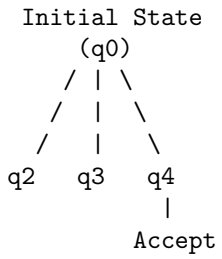
For a Non-deterministic TM

Single Tape or Multi-tape TM

Let's assume a non-deterministic TM is given to accept a string, say: $w = a_1 a_2 a_3 ... a_n$

Tree Representation

The computation of such a machine can be imagined as a **tree of possibilities**. From the **initial state**, the machine can branch into several different states depending on its choices at each step.



In this tree, there will be at least one path that leads to the **accepting state** (if the string is actually accepted by the machine). If there is no such path, then the string would not have been recognized.

Thus, the acceptance of a string depends only on the existence of at least one valid accepting path in this computational tree.

Traversal of the Tree

To process this tree, we must traverse it in some order.

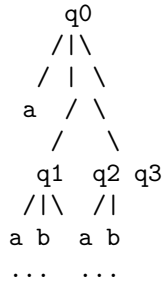
- For example, using DFS (Depth-First Search), the traversal might get stuck in a
- For example, using **DFS (Depth-First Search)**, the traversal might get stuck in a never-ending path if an infinite branch exists.
- Hence, the choice of traversal method is very important to ensure that no possible accepting path is missed.

Using Multiple Tapes

To simulate the working of this non-deterministic TM using a deterministic machine, we can use **multiple tapes**:

- The first tape contains the **input string w** (so that the input is always available for reference).
- The other two tapes are used to **simulate the tree** for every possible state.
  - Since there are only a finite number of options at each step, the branching can be systematically represented.
  - By maintaining the correct order of moves, the machine can explore all paths one by one.

As a result, the infinite tree corresponding to the input string w would look like this:



At least one of these branches may eventually lead to an **accepting state**. If such a branch exists, the machine accepts the string.

Simulation of Non-deterministic TM in Deterministic Way

Start with q_0. Let's assume we go to q_x, and then to a part of q_z.

```
0 2 3
```

But this would be DFS. We have to do BFS.

The setup:

- The first tape stays constant, holding the input string w.
- The second tape is used to copy and keep track of moves.

```
Tape 1: w
Tape 2: w
Tape 3: 0 1
```

We run the non-deterministic TM in a deterministic way.
This denotes how to enumerate which path to follow.

For example:

```
Tape 1 (Input): w = a1a2a3...an
Tape 2 (Copy): w
Tape 3: 0 2
```

Here, we copy from the first tape, then go to the **second path** in the implied order with q_2 path.

If the path reaches q_{accept}, we stop. Otherwise, we also go to q_3, q_4, q_5, and so on.

For example:

```
Tape 3: 0 1 1
Tape 3: 0 1 2
```

Let's say at some point we have:

```
Tape 3: 0 1 3
```

```
       q2 0
      (take read)
         |
      (q4 0) -----> q_accept
       / | \
      ... ...
```

After this, we check if the final state (q_{accept}) is reached.

This way, the non-deterministic machine is systematically simulated step by step in a deterministic manner using **BFS** order.

References

1. P. Linz, *An Introduction to Formal Languages and Automata*, 6th Edition, Jones & Bartlett Learning, 2016.